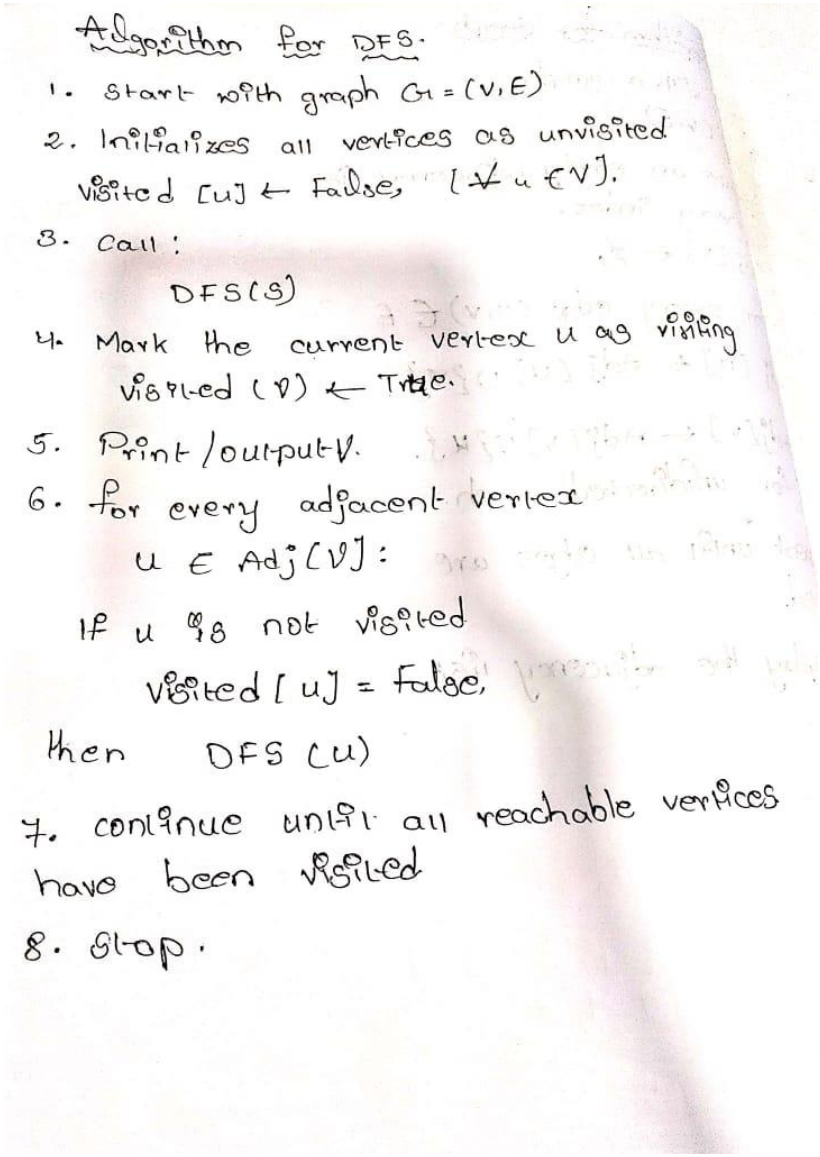
Practical: 8	Implementation of Graph and Searching (DFS and BFS).
22/09/2026	

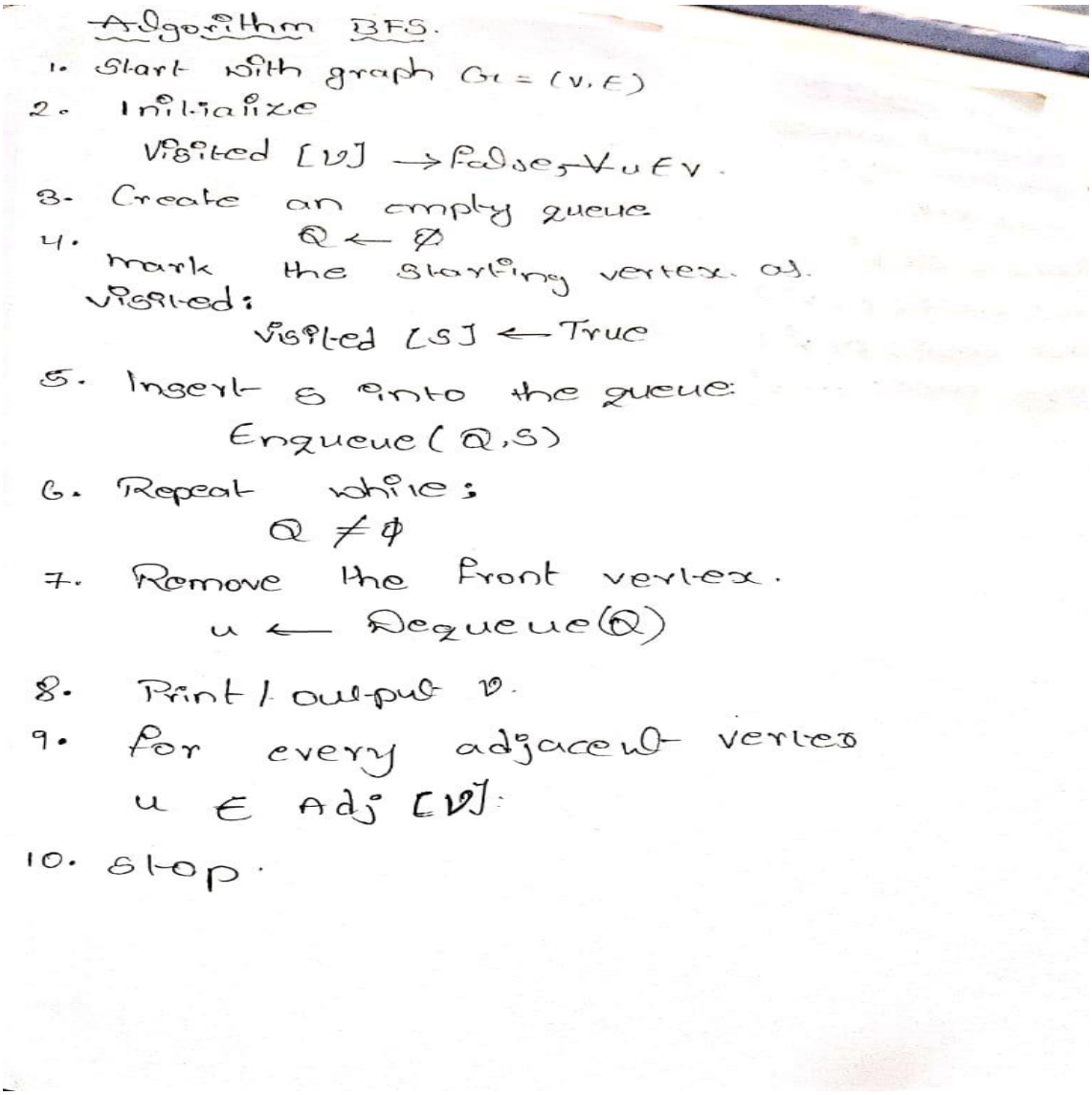
Aim: To implement of Graph and Searching (DFS and BFS).

Algorithm:



Algorithm for DFS.

1. Start with graph G = (V, E)
2. Initializes all vertices as unvisited
   visited [u] ← False, [∀ u ∈ V].
3. Call :
   DFS(S)
4. Mark the current vertex u as visiting
   visited (v) ← True.
5. Print / output V.
6. for every adjacent vertex
   u ∈ Adj[V]:
   If u is not visited
   visited [u] = False,
   then DFS (u)
7. continue until all reachable vertices have been visited
8. Stop.



Algorithm BFS.

1. Start with graph G = (V, E)
2. Initialize
   Visited [v] → False, ∀u ∈ V.
3. Create an empty queue
4. Q ← ∅
   mark the starting vertex as visited:
   Visited [s] ← True
5. Insert s into the queue:
   Enqueue (Q, s)
6. Repeat while:
   Q ≠ ∅
7. Remove the front vertex.
   u ← Dequeue(Q)
8. Print / output v.
9. For every adjacent vertex
   u ∈ Adj [v]:
10. Stop.

Program:

```
#include <iostream> #include
<queue>
using namespace std;

int main()
{
   // Static adjacency matrix
   int graph[5][5] =
```

```
    {
        {0, 1, 1, 0, 0},
        {1, 0, 0, 1, 1},
        {1, 0, 0, 0, 0},
        {0, 1, 0, 0, 0},
        {0, 1, 0, 0, 0}
    };

    int visited[5] = {0};

    // ---------- DFS ----------
cout << "DFS Traversal: ";

    int stack[5];
int top = -1;

    stack[++top] = 0;
    visited[0] = 1;

    while (top >= 0)
    {
        int vertex = stack[top--];
        cout << vertex << " ";

        // Add adjacent vertices
for (int i = 4; i >= 0; i--)
        {
            if (graph[vertex][i] == 1 && visited[i] == 0)
            {
visited[i] = 1;
                stack[++top] = i;
            }
        }
    }

    // Reset visited array
    for (int i = 0; i < 5; i++)
visited[i] = 0;

    // ---------- BFS ----------
cout << "\nBFS Traversal: ";

    queue<int> q;
```

```
    q.push(0);
    visited[0] = 1;

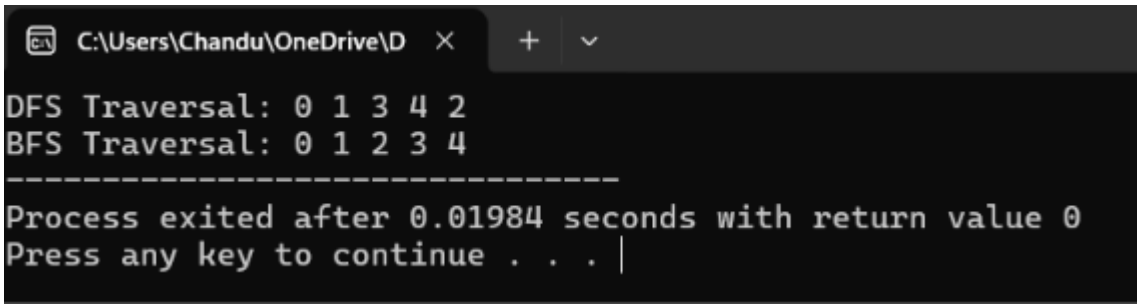
    while (!q.empty())
    {
        int vertex = q.front();
q.pop();

        cout << vertex << " ";

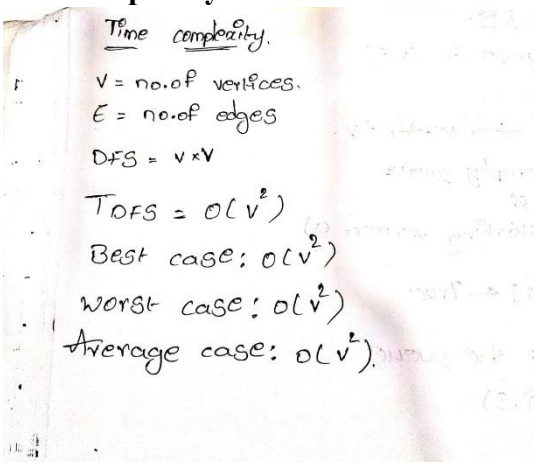
        for (int i = 0; i < 5; i++)
        {
            if (graph[vertex][i] == 1 && visited[i] == 0)
            {
                visited[i] = 1;
q.push(i);
            }
        }
    }

    return 0;
}
```

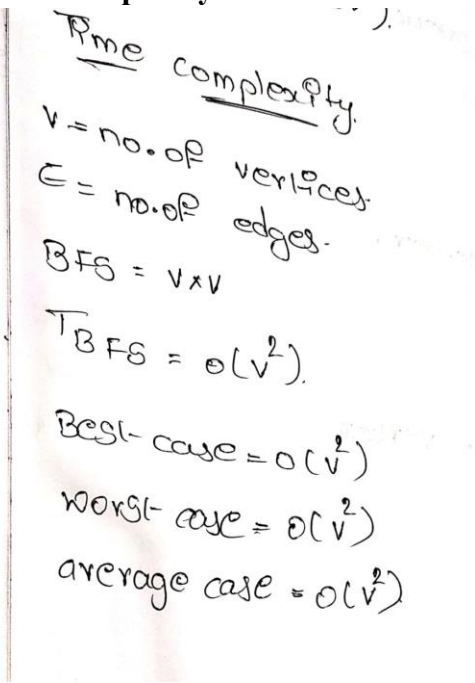
Output:



Time Complexity for DFS:



Time complexity for BFS:



Conclusion:

The Implementation of Graph and Searching DFS and BFS are successfully executed with the time complexity of $O(V^2)$
And same for all the three cases of best, average and worst.